Indoor Wireless Movement-Intelligence Pipeline

A Five-Stage System Specification

Team Roles & Interface Contracts

June 7, 2026

Abstract

This document specifies a five-stage pipeline that turns raw indoor wireless signals (UWB, BLE, Wi-Fi) into operational movement intelligence for an environment such as a hospital emergency department. Each stage is owned by one person and communicates with the next only through a strict, canonical data contract. The stages are: (1) signal standardization, (2) zone discovery via clustering, (3) movement-graph construction over time, (4) insight, anomaly, and prediction, and (5) visualization, demo, and story. Each person should treat the input and output formats below as fixed contracts so the stages compose cleanly.

Contents

Person 1 — Signal + Data Pipeline Engineer

Goal: turn raw indoor wireless readings into one exact standardized record format.

You receive raw measurements from one or more sources: UWB, Bluetooth (BLE), and Wi-Fi. The raw input may be messy and inconsistent, but your job is only to convert it into one canonical JSON record format.

Input

Your input may look like:

- a UWB range reading
- a BLE RSSI reading
- a Wi-Fi signal reading
- a timestamp

- a device ID

Output

You must output exactly one record per observation in this format:

```
{
  "timestamp_ms": 1710000000000,
  "device_id": "person_1",
  "source_type": "uwb",
  "signal_vector": [2.41, 2.87, 3.02],
  "confidence": 0.91
}
```

Exact field definitions

- `timestamp_ms`: integer milliseconds since epoch
- `device_id`: string
- `source_type`: one of `"uwb"`, `"ble"`, `"wifi"`
- `signal_vector`: numeric array, fixed length for each source type
- `confidence`: number from 0.0 to 1.0

Scope boundary

Do not add zone labels, graphs, or predictions. Your job is only to produce this standardized observation format.

Person 2 — Zone Discovery (Core ML / Clustering)

Goal: take standardized wireless observations and assign each one to a discovered zone.

You receive a stream of JSON records from Person 1 in this exact format:

```
{
  "timestamp_ms": 1710000000000,
  "device_id": "person_1",
  "source_type": "uwb",
  "signal_vector": [2.41, 2.87, 3.02],
  "confidence": 0.91
}
```

Your job is to cluster these observations into zones without using any floor plan or predefined room labels.

Important: if you receive two or more observations for the same device at the same timestamp, but from different sources such as UWB, BLE, or Wi-Fi, they should be treated as one combined piece of information for that moment, not as separate independent events. Those simultaneous measurements are still useful, but they belong together as one multimodal observation.

Output

You must output two things.

1. **One assignment record per combined observation moment**

```
{
  "timestamp_ms": 1710000000000,
  "device_id": "person_1",
  "zone_id": "zone_3",
  "zone_confidence": 0.84
}
```

2. **One zone definition record for each discovered zone**

```
{
  "zone_id": "zone_3",
  "prototype_vector": [2.38, 2.79, 3.05],
  "zone_size": 42
}
```

Exact field definitions

- `timestamp_ms`: integer milliseconds since epoch
- `device_id`: string
- `zone_id`: string such as `"zone_1"`, `"zone_2"`, etc.
- `zone_confidence`: number from 0.0 to 1.0
- `prototype_vector`: numeric array representing the cluster center
- `zone_size`: integer count of combined observation moments assigned to the zone

Scope boundary

Do not output graphs, bottlenecks, or predictions. Your job is only zone assignment and zone definitions.

Person 3 — Movement Graph Builder

Goal: take zone assignments over time and turn them into movement structure.

You receive zone assignment records from Person 2 in this exact format:

```
{
  "timestamp_ms": 1710000000000,
  "device_id": "person_1",
  "zone_id": "zone_3",
  "zone_confidence": 0.84
}
```

Your job is to build movement structure from the order of zone visits over time. This means you should keep track not only of which zones connect to which, but also how those transitions change across time, so later stages can detect evolving patterns.

Output

You must output exactly one graph summary object in this format:

```
{
  "nodes": ["zone_1", "zone_2", "zone_3"],
  "edges": [
    {
      "from_zone_id": "zone_1",
      "to_zone_id": "zone_3",
      "transition_count": 18,
      "transition_probability": 0.72
    }
  ],
  "zone_stats": {
    "zone_1": {
      "avg_dwell_ms": 4200,
      "visit_count": 31
    },
    "zone_3": {
      "avg_dwell_ms": 12700,
      "visit_count": 24
    }
  },
  "time_windows": [
    {
      "window_start_ms": 1710000000000,
      "window_end_ms": 1710000300000,
      "window_graph": {
        "nodes": ["zone_1", "zone_2"],
        "edges": [
          {
            "from_zone_id": "zone_1",
            "to_zone_id": "zone_2",
            "transition_count": 6,
            "transition_probability": 0.60
          }
        ]
      }
    }
  ]
}
```

Exact field definitions

- `nodes`: array of zone ID strings
- `edges`: array of transition objects
- `from_zone_id`: string
- `to_zone_id`: string
- `transition_count`: integer
- `transition_probability`: number from 0.0 to 1.0
- `zone_stats`: object keyed by zone ID

- `avg_dwell_ms`: integer milliseconds
- `visit_count`: integer
- `time_windows`: array of time-based snapshots of the graph
- `window_start_ms`: integer milliseconds since epoch
- `window_end_ms`: integer milliseconds since epoch
- `window_graph`: a graph object with the same basic node/edge structure as above, but for that time window only

Scope boundary

Do not output raw signals or zone clustering results. Your job is only the graph and zone movement statistics, including how they evolve over time.

Person 4 — Insight, Anomaly, and Prediction Engine

Goal: take the movement graph and produce meaningful operational insight, especially anomalies.

You receive one graph summary object from Person 3 in this exact format:


```
{
  "nodes": ["zone_1", "zone_2", "zone_3"],
  "edges": [
    {
      "from_zone_id": "zone_1",
      "to_zone_id": "zone_3",
      "transition_count": 18,
      "transition_probability": 0.72
    }
  ],
  "zone_stats": {
    "zone_1": {
      "avg_dwell_ms": 4200,
      "visit_count": 31
    },
    "zone_3": {
      "avg_dwell_ms": 12700,
      "visit_count": 24
    }
  },
  "time_windows": [
    {
      "window_start_ms": 1710000000000,
      "window_end_ms": 1710000300000,
      "window_graph": {
        "nodes": ["zone_1", "zone_2"],
        "edges": [
          {
            "from_zone_id": "zone_1",
            "to_zone_id": "zone_2",
            "transition_count": 6,
            "transition_probability": 0.60
          }
```

```
        ]
      }
    }
  ]
}
```

Your job is to detect anomalies, bottlenecks, and likely future congestion. Because you receive time-windowed graph snapshots, you should also look for changes over time, such as a zone gradually becoming more crowded, a transition pattern becoming unusual, or an area that starts behaving differently from earlier windows.

Output

You must output exactly one array of insight objects in this format:

```
[
  {
    "zone_id": "zone_3",
    "insight_type": "bottleneck_risk",
    "message": "Zone 3 is likely to become congested soon.",
    "confidence": 0.77
  }
]
```

Allowed `insight_type` values

- `"bottleneck_risk"`
- `"anomaly"`
- `"high_dwell_zone"`
- `"unexpected_transition"`
- `"congestion_forecast"`

Exact field definitions

- `zone_id`: string
- `insight_type`: one of the allowed strings above
- `message`: human-readable string
- `confidence`: number from 0.0 to 1.0

Scope boundary

Do not output raw observations, zone assignments, or graph construction. Your job is only insight and prediction, including time-based changes and emerging patterns.

Person 5 — Visualization + Demo + Story

Goal: take the graph and insight outputs and turn them into a clear, polished demo.

Input

You receive:

1. the graph summary object from Person 3
2. the insight array from Person 4

The exact inputs you will receive are as follows.

Graph summary


```
{
  "nodes": ["zone_1", "zone_2", "zone_3"],
  "edges": [
    {
      "from_zone_id": "zone_1",
      "to_zone_id": "zone_3",
      "transition_count": 18,
      "transition_probability": 0.72
    }
  ],
  "zone_stats": {
    "zone_1": {
      "avg_dwell_ms": 4200,
      "visit_count": 31
    }
  },
  "time_windows": [
    {
      "window_start_ms": 1710000000000,
      "window_end_ms": 1710000300000,
      "window_graph": {
        "nodes": ["zone_1", "zone_2"],
        "edges": [
          {
            "from_zone_id": "zone_1",
            "to_zone_id": "zone_2",
            "transition_count": 6,
            "transition_probability": 0.60
          }
        ]
      }
    }
  ]
}
```


Insight array


```
[
  {
    "zone_id": "zone_3",
    "insight_type": "bottleneck_risk",
    "message": "Zone 3 is likely to become congested soon.",
    "confidence": 0.77
  }
]
```

What to show

Your job is to display this data clearly. Because the earlier stages now include time windows, your visualization should also show how the building changes over time, not just a single static snapshot. That means the demo should make it easy to see:

- discovered zones as nodes
- transitions as directed edges
- bottlenecks or anomalies highlighted visually
- short insight messages shown on screen
- how the pattern changes across time windows
- a simple hospital-emergency story for judges

Scope boundary

Do not recompute the ML, graph, or predictions. Your job is only to present the outputs in a clean, understandable way, and to let the viewer see the evolution of the hospital over time.